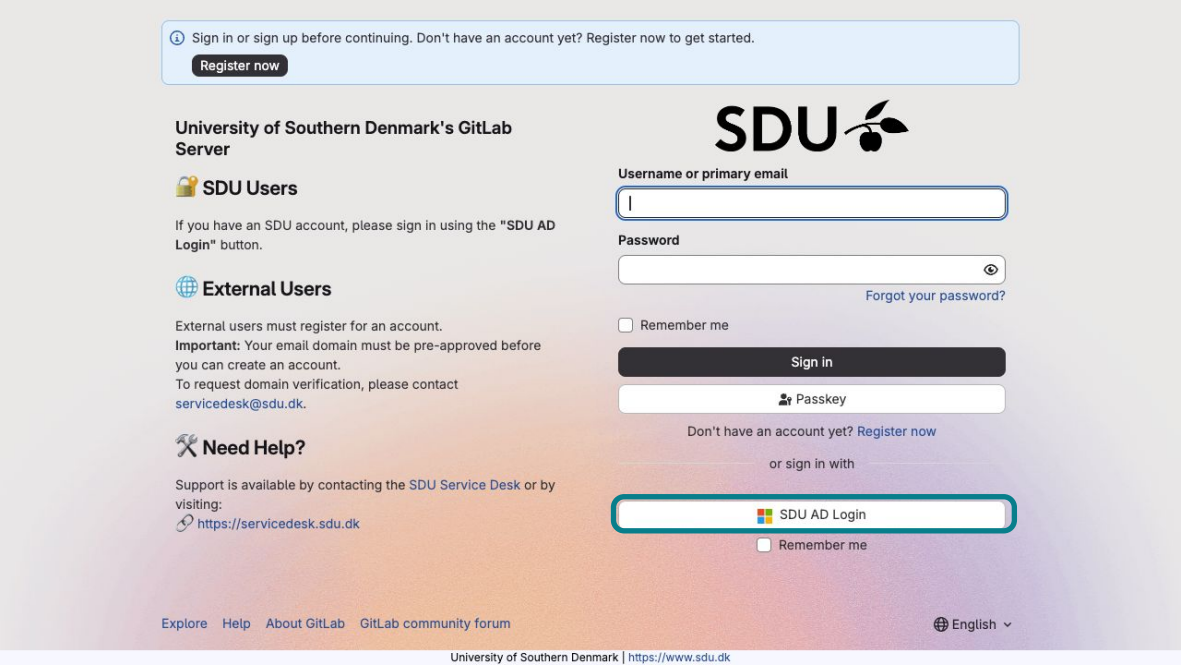
Sign in and register

Start here: sign in, find your username, and request your private assignment project.

1. Sign in to SDU GitLab

Open gitlab.sdu.dk. SDU account holders should choose **SDU AD Login**, highlighted below, and follow university sign-in. External accounts follow the approved external-user login process.



2. Find your GitLab username

After signing in, click your **avatar** (profile picture) in the upper-right corner, then click your **name or username** to open your profile. Your username may appear with an @; leave that symbol out of the form.

Copy the address of your **own profile page** from the browser. For example, if your profile is **https://gitlab.sdu.dk/example123**, enter:

```
GitLab username:    example123
GitLab profile URL: https://gitlab.sdu.dk/example123
```

This is an example only: replace both entries with your own details. Enter your full name and SDU email in their separate fields. Do not paste a course group/project URL, your display name, password or token into the username field.

3. Register for your private project

Click here to submit your details

Please submit the form **as soon as possible**. Then wait for the instructor to confirm that your private project has been assigned to you.

In the meantime, set up Python and Git as explained on page 2 and get familiar with basic Git commands.

Prepare your working environment

Prepare your software while waiting for your private project to be assigned.

4. Prepare your terminal, editor, Python and Git

macOS: open Terminal. **Windows:** open PowerShell (or Git Bash if already installed). **Linux:** open Terminal. Use a code editor that saves plain-text .py files, such as VS Code or IDLE. Do not use Word.

Check Python 3.10 or newer and Git

```
git --version
```

On macOS/Linux, check Python with:

```
python3 --version
```

On Windows, use:

```
py -3 --version
```

You should see a Git version and **Python 3.10.x or a later Python 3 version**. If a command is not found, use the installation links below, reopen your terminal, and check again. Do not type commands at Python's **>>>** prompt; type **exit()** first.

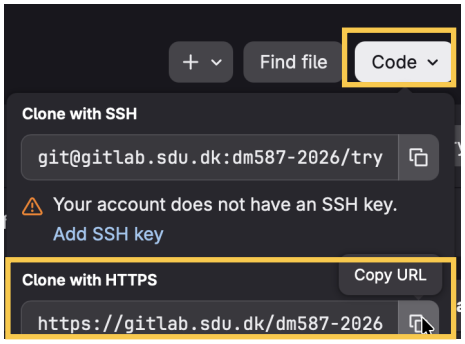
If installation is needed

Python: use python.org/downloads and follow the instructions for your operating system. Windows may use the Python install manager to install a Python 3 runtime. On Linux, use your distribution's supported Python 3 package. **Git:** use git-scm.com/install.

Use Python 3.10 or newer within the Python 3 series; Python 2 is not supported. The tryout requires no extra packages.

5. Open the private project assigned to you

Wait for the instructor's confirmation and your assigned private project URL. Open it and check that **asg-tryout/tryout.py** is present before continuing. An empty project is not ready. If access or starter files are missing, contact the instructor; do not create a replacement project or clone autograding-admin.



Actual student screenshot: click **Code**, then copy the URL under **Clone with HTTPS**. The yellow boxes mark what to use. Your own project URL will differ; do not copy the tryout-test example.

Create and use your own token

Website sign-in and Git authentication in your terminal are separate.

Create a personal access token once

Sign in to **gitlab.sdu.dk**. Click your avatar, select **Edit profile**, then open **Access > Personal access tokens**. Click **Generate token**; if a token-type menu appears, select **Legacy token** to use the scopes below. Menu labels can vary by GitLab version.

Give it a name such as **DM587 coursework**, choose an expiry covering your submission period within the university's allowed limit, and select **write_repository**. This scope includes repository read and write access: a separate read_repository selection is unnecessary. You do not need **api** permission for cloning and pushing.

Generate the token and copy the **secret token value** when it is displayed. The token name is only a label and cannot authenticate you. Store the value privately, for example in your password manager. GitLab does not show the value again after you leave that page.

Use the token when Git asks for a password

Run the clone command on page 4. If Git prompts you, enter:

```
Username: YOUR_GITLAB_USERNAME
Password: PASTE_YOUR_OWN_TOKEN_HERE
```

The two lines above explain the prompts; they are **not commands**. Paste your actual token at the password prompt and press Enter. No characters or dots may appear while typing or pasting; that is normal. Do not use your SDU website password for this token-based workflow.

Keep the token private
Never put it in the Microsoft Form, assignment code, a clone URL, email or screenshots. Students do not need the instructor's token, the runner token or a tokens.json file.

When pushing later

Use the same username/token if Git asks again. Your system may remember them securely and skip the prompts. A working SSH or credential-manager setup can also be used; the illustrated course route uses HTTPS.

If authentication fails

Check that you copied the secret value, that it has not expired, and that it has write_repository permission. An expired or lost token must be replaced. If an old credential was saved, your credential manager may need updating. Contact the instructor if access is still denied; a token does not grant access to a project you have not been invited to.

Official reference: GitLab personal access tokens. No real token values are shown in this guide.

Create your working copy

Clone once. Afterwards, keep using the same folder.

6. Run these commands on your own computer

In your terminal, first go to a folder where you want to keep coursework. Replace the placeholder with the URL copied from GitLab, keeping the quotes. Run each line separately:

```
git clone "PASTE_HTTPS_CLONE_URL_HERE" dm587-work
cd dm587-work
git status
```

If Git asks for credentials, enter your GitLab username and paste your own token at the password prompt, as explained on page 3. The password input may be invisible.

```
Receiving objects: 100% (7/7), done.
```

Actual student terminal: Receiving objects ... done confirms the clone completed. This guide calls the local folder dm587-work; the student screenshot used a different folder name.

7. Set your commit identity once

Inside dm587-work, replace these examples with your name and email:

```
git config user.name "Your Full Name"
git config user.email "your.email@example.com"
```

These identify your commits; they are not your password. Keep the submission branch provided by the instructor. If you already cloned the project, open that checkout instead of making another copy.

8. Edit the assignment file

Illustration: edit the file INSIDE your cloned project

```
dm587-work/
  README.md
  asg-tryout/
      tryout.py
```

Save your solution here

Other course files may also be present. Keep all existing names and folders.

Open **asg-tryout/tryout.py**. Replace each **pass** with your implementation. Keep the filename, folder, function names, and parameters unchanged. Do not edit .gitlab-ci.yml if present. Save the file.

Read the assignment instructions and function docstrings, then write **your own solution**. Implement every stated requirement, including required exceptions. Return values when requested; printing alone is not a return value. For later assignments, use the filenames and checking commands announced by the instructor.

Understand the test result

Run this check on your computer after saving asg-tryout/tryout.py.

9. Run the local checks from the repository root

macOS/Linux:

```
python3 -m doctest -v asg-tryout/tryout.py
```

Windows:

```
py -3 -m doctest -v asg-tryout/tryout.py
```

If your solution passes the local checks

For the current tryout file, if your answers pass all five embedded examples, the terminal ends with:

```
5 passed and 0 failed.
Test passed.
```

If your solution fails a local check

For example, if one output differs from its required result, the terminal identifies the call and compares **Expected** with **Got**. A tryout failure can look like:

```
Failed example:
    squares()
Expected:
    [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Got:
    [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

4 passed and 1 failed.
***Test Failed*** 1 failures.
```

This output is an illustration of a possible mistake, **not an instruction to submit an incorrect solution**. Your failing call, values and number of failures depend on your code. Read the requirement, correct your implementation, save and run the checks again.

Local checks and online grading
Both follow the same assignment requirements, but online grading includes additional tests. Passing all local examples does **not guarantee full marks**. For example, online tryout grading checks unequal-length lists, which the embedded examples do not cover. Local checks show no score. After pushing your own committed solution, read your actual online score in GitLab (pages 7-8).

Commit, push, then verify

Run the commands inside your cloned repository folder, on your own computer.

SAVE (file on your computer) > COMMIT (local revision) > PUSH (upload to GitLab)

10. Upload your solution

Review your changes, select only the solution file, record a revision, and upload it:

```
git status
git diff -- asg-tryout/tryout.py
git add asg-tryout/tryout.py
git commit -m "Submit tryout solution"
git push -u origin HEAD
```

HEAD means your current branch. Stay on the branch your instructor provided. A failed push is not a successful submission, even if the commit succeeded.

If push says "fetch first" or "non-fast-forward"

GitLab has newer commits. After committing your changes, run **git status**. If the working tree is clean, pull those commits and rebase your local commits onto them, then retry:

```
git pull --rebase origin main
git push -u origin HEAD
```

Use your assigned branch instead of **main** if different. If conflicts appear, stop and ask the instructor; do not push until resolved. These commands do not fix login or access errors. Do not force-push.

11. Confirm the code reached GitLab

Refresh your private project in the browser. Open **asg-tryout > tryout.py** and check that the displayed code is the version you intended to upload.

```
git rev-parse --short HEAD
```

This prints the identifier of your latest local commit. Compare it with the latest commit shown on GitLab so you know you are looking at the same revision.

Illustration: what to compare after a successful push

asg-tryout / tryout.py

Latest commit: [your commit identifier] [your commit message]

```
def squares():
    ... your saved implementation ...
```

The picture is an illustration of the comparison, not evidence that a submission has already succeeded. Your actual commit message, identifier, and code will differ.

12. Upload a correction

Edit, save, rerun the local check, then repeat step 10 with a new commit message such as **Fix tryout squares**. Confirm the new revision is visible on GitLab.

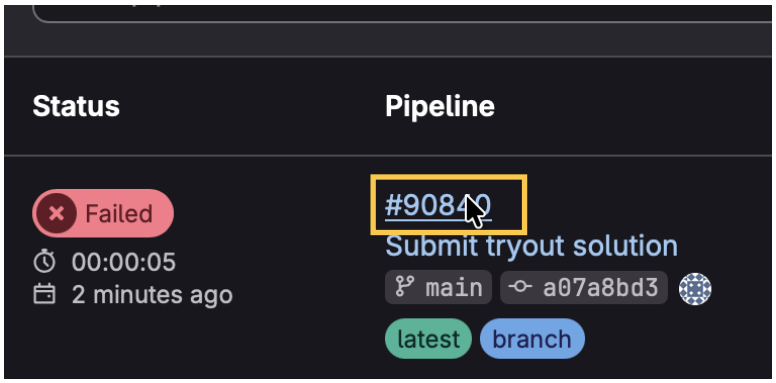
Open the job after pushing

Grading starts automatically after a successful push of new commits.

Push uploads; GitLab grades
You do not launch asg_tryout or generate its log yourself. Wait for the pipeline to finish, then open the existing job. Your terminal confirms the upload; the grade appears in GitLab. Everything up-to-date means no new commits were uploaded.

13. Open the latest pipeline

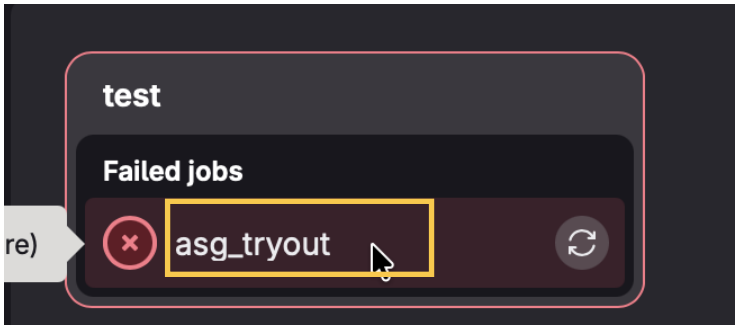
In your assigned project, select **Build > Pipelines** (or the pinned **Pipelines** entry). Find the pipeline for your latest submitted commit and click its pipeline number. The commit identifier should match the revision you pushed.



Actual student screenshot: a red Failed badge can mean tests found an incorrect answer. Open it to read the result; do not create a New pipeline.

14. Open asg_tryout

Click the **asg_tryout** job name in the pipeline diagram. Click the name, not the circular retry button. This opens its existing log.



The same route works when the job is green and Passed. If it is Pending, wait for a runner. If it remains stuck or no pipeline appears after an actual push, send the instructor the project link, commit identifier and error text.

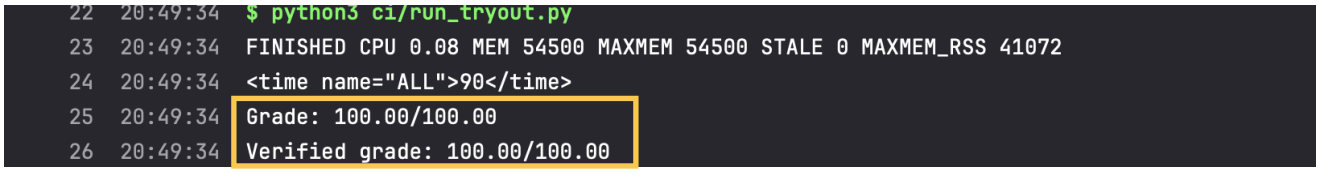
These screenshots show a test project and example commit identifiers. Use your own private project and latest submission. Continue to page 8 to read the score.

Find your score and failed checks

The highlighted lines show the result. You do not need to understand the setup messages.

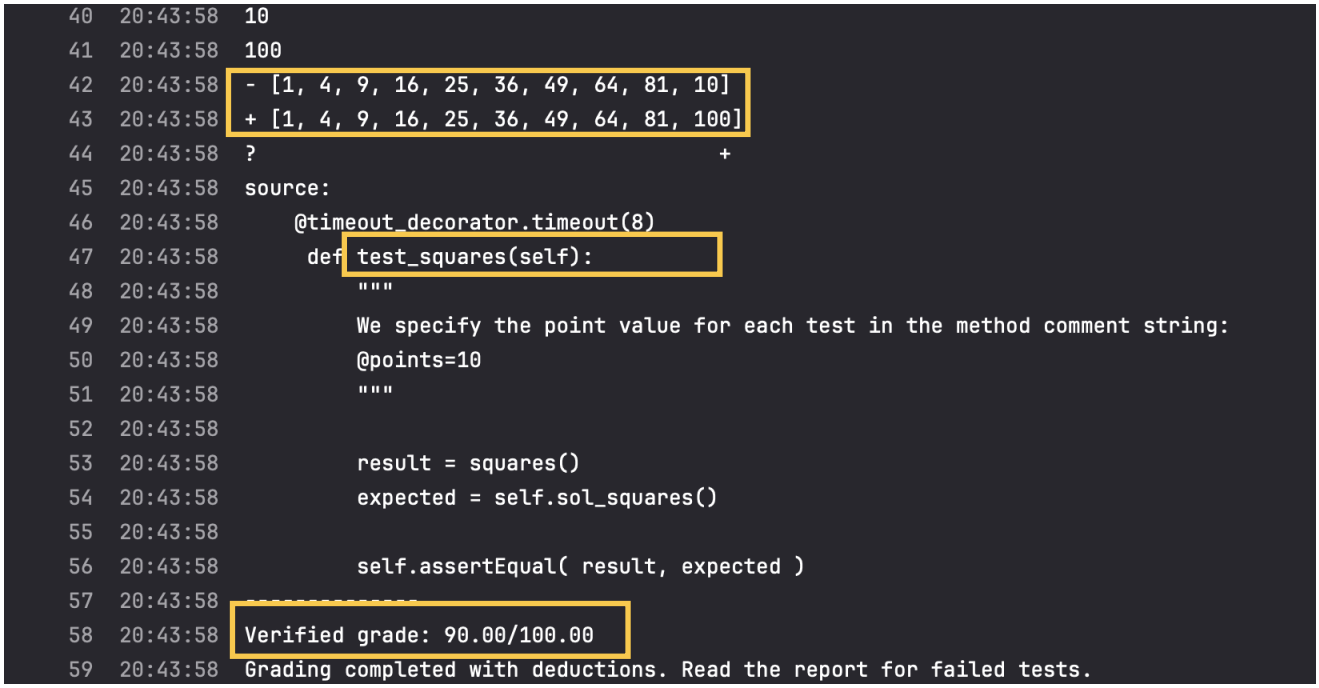
If your submission passes all grading tests

Look for **Grade** or **Verified grade**. If your solution passes all tryout grading tests, the score is 100.00/100.00 and the job is Passed, as in this screenshot.



If your submission fails a particular test

This screenshot illustrates a submission that failed **test_squares**, losing 10 points and scoring 90/100 because its other tests passed. The job is Failed, but grading completed. The boxes mark the output difference, test name and score. **Your actual score and failures depend on your own submission.**



In this comparison, the line starting with **-** is the returned result and the line starting with **+** is the expected result. Earlier lines in the full log provide the failure message. Read the reported check and correct your solution; you do not edit the grading tests.

A failed job needs its explanation
A wrong answer or Python error can fail a job. A Grading setup error instead means grading could not complete: contact the instructor. Do not treat a setup error as a valid numeric grade.

Save your correction, run the local checks, commit and push again. Open the new pipeline for that new commit. The old log keeps its old result; it does not change when you fix your code.

Other ways to read feedback

The job log is the main route; a downloadable summary is also available.

Alternative: View grade and feedback

On the job page, the external link **View grade and feedback** opens the score summary. It may require further sign-in or authorization on the university service. If that becomes confusing, use the job log on page 8; opening this summary is not required.

If the link downloads **summary.html**, open that file in your browser. Alternatively, select **Job artifacts > Browse > feedback/summary.txt** for a plain-text report, or download the artifacts and open summary.html locally. The summary lists your score, commit identifier and failed-check details; setup errors show **Score unavailable**.

Before your next work session

Open your existing checkout. If **git status** says your working tree is clean, receive instructor updates with:

```
git status
git pull --ff-only
```

Then edit, save, test, commit, push, and verify. If Git cannot update cleanly, keep your work and contact the instructor; do not reset or delete it.

If something goes wrong

Message or symptom	What you should do
python3 / py not found	Complete Python installation, reopen the terminal, and check its version.
Not a git repository	Move into your cloned dm587-work folder before using Git.
Cannot find tryout.py	Check the path and working folder; keep asg-tryout/tryout.py unchanged.
Nothing to commit	Save in the editor and use git status to see whether the intended file changed.
Authentication failed	Check your own account/token. If access is still denied, contact the instructor.
Push rejected or conflict	For fetch first/non-fast-forward, see page 6. For conflicts or access errors, ask the instructor.
Expected differs from Got	Fix the function result, save, and run the local check again.

When asking for help, include your project link, commit identifier, and relevant error text. Remove passwords and tokens. Your instructor handles project access and submission permissions.